

File Systems Forensics Assignment

December 2025

Introduction

For this assignment, required for analysis were two evidence items. In the first part, the partition table of my own laptop was to be decoded. It was manually decoded from hex values, and the results were then verified by using the sleuth kit. The other parts of the assignment were based around examining the image provided on Moodle called FSF-Asgn2-25.dd. After decoding the partition table of my laptop, the next step was to pinpoint and manually decode the hex of the \$MFT entry and give an explanation of the MFT in an NTFS file system. After doing this, it was time to decode two MFT entries that I chose, and this was done using TSK. I examined how big each file was, and the data units were occupied. Followed by an explanation of the attribute type and values linked to these entries. The last steps were to answer the last two questions of the brief which were to explain the five categories of data associated with a filesystem and identifying the major data structures associated with each category and their function (in terms of NTFS) then identifying where they can be found in the image that was provided and explaining how they compare with a FAT filesystem. The entire process of the assignment from the first part to the last is explained and documented in this report including what I did, why I did it, how I did it, and the tools I used to do it.

Tools used: the tools used in this assignment included the sleuth kit (mmls, fsstat, fls, istat) for filesystem interrogation with Linux utilities (dd, hexdump and fdisk) for raw data extraction and manual verification and fsstat for the manual hex decoding of MFT structures.

Decoding the partition table of my laptop

I am using an ubuntu Linux virtual machine to decode the partition table for this assignment.

```
ruby@ruby:~$ sudo fdisk -l
```

I used the sudo fdisk-l command to identify where my disk was mounted as the first step.

```
Disk /dev/vda: 64 GiB, 68719476736 bytes, 134217728 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: C02AB7E5-C1BE-4AAA-AC94-319D58F0558D
```

Device	Start	End	Sectors	Size	Type
/dev/vda1	2048	2203647	2201600	1G	EFI System
/dev/vda2	2203648	6397951	4194304	2G	Linux filesystem
/dev/vda3	6397952	134215679	127817728	60.9G	Linux filesystem

After the command was run, I found the /dev/vda disk which displays the disk label type. Which in this case is gpt. It is the main disk of my virtual machine and is the disk that is going to be used to fulfill the assignments requirements for manually decoding the partition table using hex values.

```
ruby@ruby:~$ sudo dd if=/dev/vda bs=512 count=1 | xxd
```

Because I'm using a Linux machine, I used this command first, to retrieve the first sector of the disk. But I switched to this command:

```
ruby@ruby:~$ sudo dd if=/dev/vda bs=512 count=1 | hexdump -C
```

The reason I made the switch to this command and used it as my preferred option is because it was easier for me to manually decode the hex values because it gets straight to

the point and I overall just found it easier to navigate what I was doing. I found this command on the website superuser.com

```
ruby@ruby:~$ sudo dd if=/dev/vda bs=512 count=1 | hexdump -C
00000000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....
.....|
*
000001c0  02 00 ee ff ff ff 01 00 00 00 ff ff ff 07 00 00  |.....
.....|
000001d0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....
.....|
*
000001f0  00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa  |.....
...U.|
1+0 records in
1+0 records out
512 bytes copied, 7.9458e-05 s, 6.4 MB/s
00000200
ruby@ruby:~$
```

After running the command `sudo dd if=/dev/vda bs=512 count=1 | hexdump -C`, the first 512 bytes of the disk `/dev/vda` were displayed. This sector corresponds to LBA 0 and contains the Master boot Record or MBR. The hexdump displays that the boot code region mostly contains zeros, and the partition table holds a single entry, which is typical of a GPT protective MBR. This shows that the disk uses the GPT partitioning scheme. The partition type value of GPT disks usually uses the 0xEE partition type value in the protective MBR, but the hex values that I manually decoded showed 0xFF instead. 0xFF is a non-standard value but because the end of the sector has 0x55AA, it confirms that the MBR is valid. On GPT disks, the actual partitioning information is stored in the GPT header at LBA 1, and the GPT partition entry array begins at LBA 2 as opposed to the MBR itself.

Manually decoded hex values table

Address	Hex Value	Description
0x1BE (0)	02	Not bootable
0x1BF – 0x1C1 (1-3)	00 EE FF	CHS start
0x1C2 (4)	FF	Partition type (non-standard value for GPT protective MBR) usually it's 0xEE

0x1C3 – 0x1C5 (5-7)	FF FF 01	CHS end
0x1C6 – 0x1C9 (8-11)	01 00 00 00	LBA start
0x1CA – 0x1CD (12-15)	FF FF 07 00	Number of sectors
0x1FE – 0x1FF	55 AA	MBR signature

My table shows the manually decoded values from the first MBR partition entry, which starts at address 0x1BE and ends at 0x1CD. The disk uses GPT (confirmed by running `sudo fdisk -l`), so the MBR exists only as a protective MBR. Because of this, several of the hex values in the entry are placeholders or obsolete fields that are not used by GPT.

TSK verification using the sleuth kit

After manually decoding the partition of my laptop in hex values, I used this command:

```
ruby@ruby:~$ sudo mmls /dev/vda
```

to verify the results. The reason for using this command was to compare the manual decoding with the version produced by the sleuth kit. The output of the `mmls` command gave a structured partition table. The table identifies GPT metadata like the GPT header and partition table, unallocated areas, and the defined partitions.

Here is the resulting output:

```

      Slot      Start      End      Length      Description
000:  Meta      0000000000  0000000000  0000000001  Safety Table
001:  - - - - -  0000000000  0000002047  0000002048  Unallocated
002:  Meta      0000000001  0000000001  0000000001  GPT Header
003:  Meta      0000000002  0000000033  0000000032  Partition Table
004:  000      0000002048  0002203647  0002201600  EFI System partit
ion
005:  001      0002203648  0006397951  0004194304  Linux filesystem
data
006:  002      0006397952  0134215679  0127817728  Linux filesystem
data
007:  - - - - -  0134215680  0134217727  0000002048  Unallocated
ruby@ruby:~$
```

This gives confirmation of the locations, lengths, and description of the partitions that were manually decoded. The GPT metadata entries and EFI system partition show that the disk uses the GUID partition table format.

Task 1 conclusion

In conclusion, part one of the assignment, which was decoding the partition table of my laptop, manually decoding the table from hex values and verifying the results using the sleuth kit. It was confirmed that the disk uses the GPT partitioning format. The protective MBR contained placeholders and obsolete values. The GPT structures stored actual partition metadata, and the results were verified by using the mmls tool from the sleuth kit which confirmed the partition layout of my manual decoding.

The concept of the MFT in an NTFS file system

The concept of the MFT in an NTFS file system is that the MFT is the central component of the NTFS file system. The reason being that it holds records for all files and directories. Each file, folder, and system has at least one entry in the MFT. They are 1KB in size. The first part of the entry has the standard MFT record header, which has 42 bytes with a fixed structure. The rest of the bytes hold attributes; these are small data structures with a precise purpose, for example one attribute can be used for storing the name of a file, and the next attribute can be used to store the content of a file. (Brian carrier, 2005)

Locating and manually decoding the hex of the \$MFT entry

To start the process of locating and manually decoding the hex of the \$MFT entry I used:

```
ruby@ruby:~/Downloads$ fls NTFS.dd
```

. I used fls to list files in the NTFS image. The output displays both NTFS system files and user files which are all stored as entries in the MFT.

```

ruby@ruby:~/Downloads$ fls NTFS.dd
r/r 4-128-1:      $AttrDef
r/r 8-128-2:      $BadClus
r/r 8-128-1:      $BadClus:$Bad
r/r 6-128-1:      $Bitmap
r/r 7-128-1:      $Boot
d/d 11-144-2:     $Extend
r/r 2-128-1:      $LogFile
r/r 0-128-1:      $MFT
r/r 1-128-1:      $MFTMirr
r/r 9-128-2:      $Secure:$SDS
r/r 9-144-3:      $Secure:$SDH
r/r 9-144-4:      $Secure:$SII
r/r 10-128-1:     $UpCase
r/r 3-128-3:      $Volume
r/r 64-128-2:     picture10.JPG
r/r 65-128-2:     picture9.jpg
V/V 66: $OrphanFiles

```

File listing using fls validated the functionality of the MFT, showing 66 entries with system metadata and user files. For part 3 of the brief, I chose the entries 64 picture10.JPG and entry 65 picture9.jpg to decode which are user files that store their data outside of the MFT. Which would be displayed as non-resident in the \$DATA attribute when analyzing the outputs. This selection enables the practical examination of NTFS storage mechanisms like cluster run encoding, allocated size compared to actual size tracking, and resident compared to non-resident attribute management.

To find where the MFT begins, the filesystem type and version, sector size, cluster size the total clusters/sectors and all the NTFS attribute types I used fsstat

```

ruby@ruby:~/Downloads$ fsstat -o 0 NTFS.dd

```

the

–o 0 specifies a zero-sector offset to show that the image contains a raw NTFS filesystem without a partition table wrapper. I took this approach because at first, I started with mmls

NTFS.dd but it resulted in no output which was confirmation there was no partition structure. The entire file contains the NTFS filesystem starting at byte 0.

These are the specific parts I took from the fsstat output:

File system type	NTFS
Cluster size	4096 bytes
First cluster of MFT	4
Size of MFT entries	1024 bytes

To analyze the \$MFT entry, I used `ruby@ruby:~/Downloads$ istat -o 0 NTFS.dd 0`. This command extracted and interpreted MFT entry 0, which showed its full metadata structure which included allocation status, sequence number, attribute inventory (with types, sizes, residence), timestamps, filename etc. The output highlighted forensic artifacts like timestamp discrepancies between attributes.

The following tables summarize key findings from the istat output:

MFT Entry Header

Entry	0	This is the \$MFT file (first MFT entry).
Sequence	1	Version number.
\$Logfile sequence number	0	
Allocated File		This means the file entry is in use.
	1	This means only one directory references this file.

\$STANDARD_INFORMATION Attribute

Flags	Hidden, System	\$MFT is a system file.
Owner ID	0	Attribute data store within MFT entry and not in clusters.
Security ID	0 ()	
Created	000-00-00 00:00:00 (UTC)	Timestamps are zeros.
Size	72 bytes	

\$FILE_NAME Attribute values:

Name	\$MFT	
Parent MFT Entry	5	Root directory
Sequence	5	
Allocated size	27648	
Actual size	27648	
Created	2012-01-24 21:49:52.000000000	(UTC)
Resident, size	74 bytes	

\$DATA Attribute:

Type	\$DATA	(128 -1).
Name	N/A	
Non-Resident		Data stored outside MFT (in clusters).
Size	67584	The MFT table itself.
Init_size	67584 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 0 0	Clusters 4-20 (17 x 4096 = 69,632 bytes) this is where all other MFT entries are stored.

\$BITMAP Attribute:

Type	\$BITMAP (176-3)	Tracks which MFT entries are allocated or free.
Name	N/A	
Non-Resident		
Size	16	Track 128 entries in 8MB image.
Init_size	16	
2		Stored in cluster 2.

For decoding the hex manually, I extracted the first 256 bytes of the \$MFT entry using

```
ruby@ruby:~/Downloads$ dd if=NTFS.dd bs=1 count=256 skip=16384 | hexdump -C
```

. This command targeted MFT entry 0 by skipping 16,384 bytes (cluster 4 is the start and the cluster size = 4096 which was found with the fsstat command).

Extracted 256 bytes. Enough to capture the MFT entry header and initial attributes.

Used byte level precision (bs=1) to avoid alignment errors.

Converted raw bytes to hex/ascii.

The output provided the hexadecimal representation for manual decoding and comparison with istat's results.

The output:

```
00000000  46 49 4c 45 30 00 03 00 00 00 00 00 00 00 00 00 |FILE0.....
.....|
00000010  01 00 01 00 38 00 01 00 98 01 00 00 00 04 00 00 |....8.....
.....|
00000020  00 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00 |.....
.....|
00000030  04 00 00 00 00 00 00 00 10 00 00 00 60 00 00 00 |.....
..`...|
00000040  00 00 18 00 00 00 00 00 48 00 00 00 18 00 00 00 |.....H..
.....|
00000050  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....
.....|
*
00000070  06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....
.....|
00000080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....
.....|
00000090  00 00 00 00 00 00 00 00 30 00 00 00 68 00 00 00 |.....0..
.h...|
```

```

256+0 records in
256+0 records out
256 bytes copied, 0.00301942 s, 84.8 kB/s
000000a0 00 00 18 00 00 00 02 00 4a 00 00 00 18 00 01 00 |.....J..
.....|
000000b0 05 00 00 00 00 00 05 00 00 40 f7 19 e2 da cc 01 |.....@.
.....|
000000c0 00 40 f7 19 e2 da cc 01 00 40 f7 19 e2 da cc 01 |. @.....@.
.....|
000000d0 00 40 f7 19 e2 da cc 01 00 6c 00 00 00 00 00 00 |. @.....l.
.....|
000000e0 00 6c 00 00 00 00 00 00 06 00 00 00 00 00 00 |.l.....
.....|
000000f0 04 03 24 00 4d 00 46 00 54 00 00 00 00 00 00 |..$.M.F.T..
.....|
00000100
ruby@ruby:~/Downloads$

```

The hex dump of the \$MFT entry was manually decoded to identify the MFT record header and its attributes. The entry starts with the FILE signature which was

discovered by putting the hex values 46 49 4c 45 into a hex to string converter. The FILE discovery confirmed that it is a valid NTFS MFT record. The header field that was decoded shows a standard 1024-byte MFT entry that is displayed as in use. The offset to the first attribute shows where the attribute list begins, and attributes like \$STANDARD_INFORMATION and \$FILE_NAME were identified. The existence of the UTF-16 encoded \$MFT filename confirms that this entry represents the Master File Table (MFT) itself.

Manually decoded hex values \$MFT record header

Address	Hex Value	Description
00x00 – 0x03	46 49 4c 45	MFT entry signature (FILE)
0x04 – 0x05	30 00	Offset to fix up array
0x06 – 0x07	03 00	Total fix up entries
0x08 - 0x0F	00 00 00 00 00 00 00 00	\$Logfile sequence number
0x10 - 0x11	01 00	Sequence number
0x12 – 0x13	01 00	Hard link count
0x14 – 0x15	36 00	Offset to first attribute
0x16 – 0x17	01 00	Flags: entry in use (not a directory)
0x18 - 0x1B	98 01 00 00	Used size of MFT entry
0x1C – 0x1F	00 04 00 00	Allocated size of MFT entry (1024 bytes)
0x20 - 0x27	00 00 00 00 00 00 00 00	Base record reference
0x26 – 0x29	04 00	ID of the next attribute

Manually decoded attributes \$MFT entry

Address	Hex Value	Description
---------	-----------	-------------

0x38 – 0x3B	10 00 00 00	Attribute type: \$STANDARD_INFORMATION
-0x90 - 0x93	30 00 00 00	Attribute type: \$FILE_NAME
-0xF0	4D 00 46 00 54 00	UTF-16 filename: \$MFT

This screenshot shows the hex to string converter which confirms the hex values 46 49 4c

From: Hexadecimal To: Text

Open File Sample

Paste hex code numbers or drop file

46 49 4c 45

Character encoding: ASCII

= Convert x Reset ↕ Swap

FILE

45 = FILE:

(Rapidtables.com, 2025).

MFT files of my choice decoded using TSK

From the fls NTFS.dd command in entry 64 there was a picture10.JPG file. This is the first MFT file of my choice that I decoded using TSK.

To start, I used the following command `ruby@ruby:~$ istat -o 0 NTFS.dd 64`.

This command directly accessed the picture10.JPG file and displayed the data inside.

Below is the key findings summarized to satisfy the questions in the brief.

How big is each file?

The actual size of picture10.JPG: 92853.

The allocated size: 94,208 bytes.

Slack Space: 1,355 bytes.

What data units does it occupy?

Clusters: 350 – 372.

Total amount of clusters: 23 clusters.

Calculations: $372 - 350 + 1 = 23$.

Total Space: 94,208. (multiplied the total clusters by the total allocated size).

The attribute types and attribute values associated with the entries:

\$STANDARD_INFORMATION (0x10): Resident, 48 bytes. Contains primary timestamps such as file created: 2012-01-24 22:01:02.561286000 (UTC).

\$FILE_NAME (0x30): Resident, 92 bytes. Stores filename “picture10.JPG” and parent directory reference: entry 5. (which is the root).

\$SECURITY_DESCRIPTOR (0x50): Resident, 80 bytes. Basic windows security with ID’s containing 0.

\$DATA (0x80): Non-resident, 92,853 bytes. File content stored in clusters 350-372 with sequential data runs.

The second file MFT entry I chose to decode was the picture9.jpg file at entry 65.

The process started the same way using the command

`ruby@ruby:~$ istat -o 0 NTFS.dd 65`. just using the entry number of picture9.jpg instead of the entry for picture10.JPG.

Below is the key findings summarized to satisfy the questions in the brief:

How big is each file?

The actual size of picture9.jpg: 187035.

Allocated space: 188,416.

Slack space: 1,381 bytes.

What data units does it occupy?

Clusters: 375-418.

Total amount of clusters: 46 clusters.

Calculations: $8+8+8+8+8+6 = 64$

The attribute types and attribute values associated with the entries:

\$STANDARD_INFO (resident): created 2012-01-24 22:01:02.527098000 (UTC)

Modified: 2012-01-24 22:01:02.527098000 (UTC).

\$FILE_NAME (Resident): filename "picture9.jpg", parent directory (entry 5).

\$SECURITY_DISK (Resident) basic permissions.

\$DATA (Non-Resident): Content in clusters 373-418.

Conclusion: Through the analysis of MFT entry 64 picture10.JPG and entry 65 picture9.jpg, I demonstrated how NTFS manages file metadata and storage. I answered each question in the brief with the metadata that was specific to each of the MFT entries. Both picture10.JPG and picture9.jpg showed identical attribute structures. Both have \$STANDARD_INFORMATION, \$FILE_NAME, \$SECURITY_DESCRIPTOR and \$DATA. Since the \$DATA attributes were non-resident, it confirmed that the file contents are stored externally in allocated clusters. And it also confirmed that cluster allocation was visible in data runs. This demonstrated NTFS's efficient storage management for files of different sizes.

Explain the 5 categories of data associated with a filesystem:

1. **User Application:** Software applications that initiate file operations such as reading data, writing new content, or removing files from the system.
2. **Logical File System:** Handles the organizational layer which includes file naming, directory structure, metadata management and permission controls.
3. **Virtual File System (VFS):** Provides a unified interface that allows multiple file system types to operate seamlessly within the same environment.
4. **Physical File System:** Manages the low-level storage of data blocks across physical disk sectors and tracks.
5. **Partitions: Segments** of a storage device that contains organized file systems and data storage areas.

(geeksforgeeks.com, 2025)

In terms of NTFS, identify the major data structures associated with each category and their function, identify where these can be found in the supplied image. How do these compare with a FAT file system?

In NTFS, the five data categories correspond to specific structures, e.g., content data is stored in the \$DATA attribute (clusters 350-372 for picture10.JPG show this.), metadata is contained in MFT entries with the \$STANDARD_INFORMATION and \$FILE_NAME attributes (located starting at cluster 4), application data includes the \$LogFile (entry 2 is visible in fls output), Structural data is made up of system files like \$BITMAP. These can all be found in the supplied image (NTFS.dd) through analysis with TSK tools. Compared to FAT, NTFS uses a centralized MFT with rich attributes and employs \$Bitmap for faster free space lookup whereas FAT's sequential table scans, features journaling via \$LogFile and FAT doesn't have that. It stores security descriptors which are also not in FAT and utilizes data runs in MFT verses cluster chains in the FAT table making NTFS more efficient for modern storage needs.

Assignment conclusion

This assignment provided complete insight into filesystem forensic analysis through practical examination of partition tables and NTFS structures. The manual MBR decoding showed contemporary GPT implementations, whereas the NTFS image analysis showed MFT functionality. The comparison between NTFS's attribute rich architecture and FAT's simpler table-based design showed the filesystem advancements which are essential to forensic recovery, metadata analysis, and evidence preservation in digital investigations.

Bibliography

way (2012). *Is there a way to read the contents of the master boot record?* [online] Super User. Available at: <https://superuser.com/questions/510853/is-there-a-way-to-read-the-contents-of-the-master-boot-record> [Accessed 9 Dec. 2025].

Brian Carrier (2005). *File systems forensics analysis pdf*. Available at: <https://github.com> [Accessed 9 Dec. 2025].

RapidTables (n.d.). *Hex to ASCII | Hex to Text String Converter*. [online] www.rapidtables.com. Available at: <https://www.rapidtables.com/convert/number/hex-to-ascii.html> [Accessed 10 Dec. 2025].

GeeksforGeeks (2017). *File Systems in Operating System*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/operating-systems/file-systems-in-operating-system/> [Accessed 11 Dec. 2025].

Sleuthkit (2019). *The Sleuth Kit (TSK) & Autopsy: Open Source Digital Forensics Tools*. [online] Sleuthkit.org. Available at: <https://www.sleuthkit.org/> [Accessed 11 Dec. 2025].

